

Informe de Pruebas Unitarias y Cobertura de Código

Tecnología de Pruebas: Vitest (TypeScript/JavaScript)

portionCalcLogic

Fecha de Análisis: Junio de 2026

1. Descripción de Funciones y Lógica de Negocio

El archivo bajo análisis exporta la interfaz de datos SetData y cuatro funciones principales encargadas del cálculo:

A. calcNumberOfSets(units, unitsPerSet, ovenNumber)

Determina la cantidad de tandas de horneado (o ciclos de producción) requeridas para procesar todas las unidades solicitadas.

- **Fórmula conceptual simulada:**

$$\text{Sets} = \lceil \frac{\text{units}}{\text{unitsPerSet} \times \text{ovenNumber}} \rceil$$

- **Reglas de negocio validadas:**
- Si las unidades a preparar son menores o iguales a la capacidad total de un ciclo simultáneo de todos los hornos, el resultado debe ser 1.
- Si las unidades son 0, el resultado es 0.
- Se deben disparar errores controlados si unitsPerSet o ovenNumber son menores o iguales a 0.

B. calcTimeInOven(sets, ovenTime)

Calcula el tiempo total que los hornos estarán activos y encendidos de manera secuencial a lo largo de todas las tandas.

- **Fórmula:**

$$\text{Total Oven Time} = \text{sets} \times \text{ovenTime}$$

- **Reglas de negocio validadas:**
- Si el número de tandas (sets) es 0, el tiempo de horno es 0.
- Lanza un error si el tiempo unitario del horno (ovenTime) es negativo.

C. calcHandlingTime(handlingTime, sets, ovenNumber)

Estima el tiempo de manipulación manual (carga, descarga, preparación de bandejas) requerido por los operarios.

- **Fórmula conceptual observada:**
- Para 1 sola tanda (sets === 1): El tiempo de manipulación es exactamente el handlingTime unitario (caso base).
- Para múltiples tandas (sets > 1): El tiempo total se escala multiplicando la manipulación por el número de tandas y el número de hornos operativos.

$$\text{Total Handling Time} = \text{handlingTime} \times \text{sets} \times \text{ovenNumber}$$

- **Reglas de negocio validadas:**
- Si no hay tandas (sets === 0), el tiempo de manipulación es 0.
- Lanza un error si el tiempo de manipulación unitario (handlingTime) es negativo.

D. calcFinalOvenTime(data: SetData)

Actúa como la función orquestadora. Toma un objeto de configuración que implementa SetData y calcula el consolidado de tiempos devolviendo un reporte con los valores individuales desglosados y el tiempo final en minutos.

- **Estructura de Entrada (SetData):**

```
interface SetData {
    units: number;           // Unidades totales a cocinar
    unitsPerSet: number;     // Capacidad por set (tanda)
    ovenTime: number;        // Tiempo de horneado por set (minutos)
    handlingTime: number;    // Tiempo de manipulación (minutos)
    ovenNumber: number;      // Cantidad de hornos disponibles
}
```
- **Estructura de Salida esperada:**

```
interface FinalOvenTimeResult {
    setNumber: number;       // Tandas totales necesarias
    totalTimeOven: number;   // Tiempo total de horneado
    totalHandlingTime: number; // Tiempo total de manipulación
    finalTimeMinutes: number; // Tiempo total de proceso acumulado
                              // (horno + manipulación)
}
```

2. Matriz de Casos de Prueba

El suite de pruebas implementado valida de forma minuciosa los siguientes escenarios:

Función	Caso de Prueba	Entrada de Datos	Resultado Esperado	Tipo de Test
calcNumberOfSets	Flujo normal (50 u, cap: 10, 2 hornos)	(50, 10, 2)	3 sets	Happy Path
calcNumberOfSets	Unidades < Capacidad (5 u, cap: 10, 1 horno)	(5, 10, 1)	1 set	Edge Case
calcNumberOfSets	Unidades = Capacidad (10 u, cap: 10, 1 horno)	(10, 10, 1)	1 set	Edge Case
calcNumberOfSets	Sin unidades (0 u, cap: 10, 2 hornos)	(0, 10, 2)	0 sets	Edge Case (Cero)
calcNumberOfSets	Capacidad inválida de set (0 o negativo)	(10, 0, 2) o (10, -5, 2)	Lanza excepción (toThrow)	Robustez / Error
calcNumberOfSets	Hornos inválidos (0)	(10, 10, 0)	Lanza excepción (toThrow)	Robustez / Error
calcTimeInOven	Multiplicación secuencial estándar	(3, 45)	135 min	Happy Path
calcTimeInOven	Sin tandas (0 sets)	(0, 45)	0 min	Edge Case (Cero)
calcTimeInOven	Tiempo de horno negativo	(2, -10)	Lanza excepción (toThrow)	Robustez / Error

CalcHandlingTime	Múltiples tandas (sets > 1)	(5, 3, 2)	30 min	Happy Path
CalcHandlingTime	Única tanda (set = 1)	(5, 1, 4)	5 min	Edge Case
CalcHandlingTime	Sin tandas (sets = 0)	(5, 0, 2)	0 min	Edge Case (Cero)
CalcHandlingTime	Tiempo de manipulación negativo	(-5, 2, 2)	Lanza excepción (toThrow)	Robustez / Error
calcFinalOvenTime	Orquestación: Ruta Feliz Estándar	SetData (50u, cap:10, horno:30m, manip:5m, hornos:2)	setNumber: 3, totalTimeOven: 90, totalHandlingTime: 30, finalTimeMinutes: 120	Happy Path
calcFinalOvenTime	Orquestación: Menos unidades que tanda, 1 horno	SetData (5u, cap:10, horno:60m, manip:10m, hornos:1)	setNumber: 1, totalTimeOven: 60, totalHandlingTime: 10, finalTimeMinutes: 70	Edge Case
calcFinalOvenTime	Orquestación: Multi-hornos con sobrantes	SetData (35u, cap:10, horno:40m, manip:5m, hornos:3)	setNumber: 2, totalTimeOven: 80, totalHandlingTime: 30, finalTimeMinutes: 110	Edge Case
calcFinalOvenTime	Orquestación: 0 unidades globales	SetData (0u, cap:10, horno:30m,	setNumber: 0, totalTimeOven: 0, totalHandlingTime: 0,	Edge Case (Cero)

		manip:5m, hornos:2)	finalTimeMinutes: 0	
calcFinalOvenTime	Orquestación: Parámetro negativo inválido	SetData (10u, cap:-5, horno:30m, manip:5m, hornos:2)	Lanza excepción (toThrow)	Robustez / Error

3. Análisis de Cobertura (*Coverage*) de las Pruebas

Basándonos en la suite de pruebas unitarias implementada, se realiza la siguiente proyección técnica sobre el porcentaje de cobertura del código fuente asociado a `portionCalcLogic.ts`.

A. Cobertura de Sentencias (*Statements*): ~100%

Las pruebas cubren todas las ramas y caminos lógicos factibles de las funciones. Cada asignación de variable, bloque de retorno y llamada a funciones utilitarias es invocada directamente.

B. Cobertura de Ramas (*Branches*): ~100%

Se han diseñado casos de prueba específicos para todos los operadores de decisión lógica condicional (`if/else` y validaciones de errores):

- **Ramas de validación de argumentos:** Se introducen deliberadamente valores inválidos (negativos o iguales a cero en variables clave) que activan los bloques `throw new Error()` de validación.
- **Condicionales lógicas de negocio:**
- La bifurcación de `sets === 1` contra `sets > 1` en `CalcHandlingTime` está 100% cubierta (con pruebas específicas para `sets: 1`, `sets: 3` y `sets: 0`).
- La bifurcación de `units === 0` está cubierta tanto de manera aislada como en la integración del orquestador.

C. Cobertura de Funciones (*Functions*): 100%

Las 4 funciones exportadas por el archivo (`calcNumberOfSets`, `calcTimeInOven`, `CalcHandlingTime` y `calcFinalOvenTime`) son explícitamente importadas y probadas bajo distintos contextos.

Variables Geográficas y Ambientales

1. Descripción General

El módulo `environmentalVariablesLogic.ts` implementa la lógica del RF_08 para registrar variables geográficas y ambientales. Incluye validación de roles, verificación de recetas, validación de parámetros, cálculo de factores de ajuste, recálculo de recetas, persistencia temporal de datos y registro completo del flujo.

Funciones Principales

`checkUserRole` valida roles autorizados. `checkRecipesExist` verifica la existencia de recetas. `validateEnvironmentalParams` controla rangos permitidos.

`calcAdjustmentFactors` calcula factores de humedad, temperatura, altitud y presión. `applyEnvironmentalAdjustmentToRecipe` integra el ajuste ambiental con los cálculos de producción. `registerEnvironmentalVariables` ejecuta el flujo principal.

`getStoredEnvironmentalParams` consulta datos almacenados.

`registerAndRecalculate` combina registro y recálculo.

2. Casos de Prueba

Se ejecutaron 26 casos agrupados en 7 suites. Se validaron escenarios exitosos, condiciones límite y manejo de errores. Las pruebas verifican control de acceso, existencia de recetas, validación de rangos, cálculo de factores ambientales, persistencia de datos, integración con `portionCalcLogic` y recálculo de recetas.

Resumen de Resultados

Happy Path: 11 pruebas. Casos límite: 9 pruebas. Robustez y manejo de errores: 6 pruebas. Todos los comportamientos esperados fueron verificados satisfactoriamente.

3. Cobertura de Código

Las pruebas ejecutan la totalidad de las funciones exportadas y cubren los caminos principales del sistema. La cobertura estimada es cercana al 100 % en sentencias, ramas, funciones y líneas relevantes del módulo.

4. Integración

La integración con `portionCalcLogic` fue validada mediante pruebas que verifican el recálculo de tiempos, la coherencia de los resultados obtenidos y la correcta inclusión de los factores ambientales en los objetos retornados.

5. Persistencia

Se comprobó que los parámetros registrados permanecen disponibles para consultas posteriores mediante `getStoredEnvironmentalParams`, validando la postcondición definida para el RF_08.

6. Conclusiones

La suite demuestra un comportamiento estable del módulo. Se validan correctamente las precondiciones, reglas de negocio, cálculos ambientales, persistencia temporal e integración con otros componentes. Los resultados respaldan una cobertura funcional completa y una alta confiabilidad del proceso de registro ambiental.

Módulo ingredientLogic

Tecnología de Pruebas: Vitest (TypeScript/JavaScript) **Fecha de Análisis:** Junio de 2026

1. Descripción de Funciones y Lógica de Negocio

El archivo bajo análisis exporta las interfaces de datos y seis funciones principales encargadas del cálculo de costos de ingredientes y proporciones de hidratación.

A. computeIngredientRealCost({ costPerGram, lossPercentage }) Calcula el costo real de un ingrediente tomando en cuenta la cantidad de materia prima que se pierde (merma). Fórmula conceptual simulada:

$$RealCost = \frac{costPerGram}{1 - \left(\frac{lossPercentage}{100}\right)}$$

Reglas de negocio validadas:

- Si la merma es **0%**, el costo real devuelto es igual al costo bruto.
- Se deben disparar errores controlados si costPerGram es menor a 0, o si lossPercentage está fuera del rango permitido (menor a 0 o mayor o igual a **100%**).

B. computeBruteCostPerPound({ costPerGram }) Determina el costo bruto para una libra colombiana (**500g**) sin tomar en cuenta la merma. Fórmula:

$$CostPerPound = costPerGram \times 500$$

Reglas de negocio validadas:

- Si el costo ingresado es 0, el resultado es 0.
- Lanza un error si el costo por gramo es negativo.

C. computeBruteCostPerKilo({ costPerGram }) Determina el costo bruto para un kilogramo (**1000g**) estándar. Fórmula:

$$CostPerKilo = costPerGram \times 1000$$

Reglas de

negocio validadas:

- Si el costo ingresado es 0, el resultado es 0.
- Lanza un error si el costo por gramo es negativo.

D. computeRealCostPerPound({ costPerGram, lossPercentage }) y computeRealCostPerKilo({ costPerGram, lossPercentage })

Actúan componiendo la función base de costo real (computeIngredientRealCost) para escalar el valor ya afectado por la merma hacia proporciones estándar (libra o kilo).

Reglas de negocio validadas:

- Heredan de manera implícita las protecciones de la función base, garantizando que los datos negativos o mermas fuera de rango detengan la ejecución inmediatamente.

E. computeHydrationAmount({ hydrationPercentage }) Calcula la proporción decimal equivalente de hidratación para ser utilizada en posteriores cálculos de peso en panadería/cocina. Fórmula conceptual simulada:

$$HydrationProportion = \frac{hydrationPercentage}{100}$$

Reglas de negocio validadas:

- Lanza un error si el porcentaje de hidratación es menor a **0%** o mayor a **100%**.

2. Matriz de Casos de Prueba

El suite de pruebas implementado valida de forma minuciosa los siguientes escenarios:

Función	Caso de Prueba	Entrada de Datos	Resultado Esperado	Tipo de Test
computeIngredientRealCost	Merma inexistente (0%)	{ costPerGram: 100, lossPercentage: 0 }	100	Edge Case (Cero)
computeIngredientRealCost	Flujo normal con merma del 50%	{ costPerGram: 10, lossPercentage: 50 }	20	Happy Path
computeIngredientRealCost	Precisión de decimales flotantes	{ costPerGram: 10, lossPercentage: 33.3 }	14.9925	Edge Case (Precisión)
computeIngredientRealCost	Porcentajes inválidos (100% , negativos o > 100%)	{ lossPercentage: 100 }, { lossPercentage: 150 }, { lossPercentage: -5 }	Lanza excepción (toThrow)	Robustez / Error
computeIngredientRealCost	Costo por gramo negativo	{ costPerGram: -10, lossPercentage: 20 }	Lanza excepción (toThrow)	Robustez / Error

computeBruteCostPer Pound	Multiplicación estándar (Libra = 500g)	{ costPerGram: 5, lossPercentage: 0 }	2500	Happy Path
computeBruteCostPer Kilo	Multiplicación estándar (Kilo = 1000g)	{ costPerGram: 5, lossPercentage: 0 }	5000	Happy Path
computeBruteCost... (Ambas)	Costo base en 0	{ costPerGram: 0 }	0	Edge Case (Cero)
computeBruteCost... (Ambas)	Costo base negativo	{ costPerGram: -5 }	Lanza excepción (toThrow)	Robustez / Error
computeRealCostPer Pound	Orquestación : Costo real válido (Libra)	{ costPerGram: 10, lossPercentage: 50 }	10000	Happy Path
computeRealCostPer Kilo	Orquestación : Costo real válido (Kilo)	{ costPerGram: 10, lossPercentage: 50 }	20000	Happy Path
computeRealCost... (Ambas)	Costo real base en 0	{ costPerGram: 0, lossPercentage: 20 }	0	Edge Case (Cero)
computeRealCost... (Ambas)	Parámetro heredado negativo o fuera de rango	{ costPerGram: -10 } o { lossPercentage: 110 }	Lanza excepción (toThrow)	Robustez / Error

computeHydrationAmount	Proporción estándar	{ hydrationPercentage: 50 }	0.5	Happy Path
computeHydrationAmount	Límites permitidos (0% y 100%)	{ hydrationPercentage: 0 }, { hydrationPercentage: 100 }	0 y 1	Edge Case
computeHydrationAmount	Porcentajes fuera de rango permitidos	{ hydrationPercentage: -5 }, { hydrationPercentage: 120 }	Lanza excepción (toThrow)	Robustez / Error

3. Análisis de Cobertura (Coverage) de las Pruebas

Basándonos en la suite de pruebas unitarias implementada, se realiza la siguiente proyección técnica sobre el porcentaje de cobertura del código fuente asociado a ingredientLogic.ts.

A. Cobertura de Sentencias (Statements): ~100% Las pruebas cubren todas las ramas y caminos lógicos factibles de las funciones. Cada asignación de variable, bloque de retorno y llamada a funciones utilitarias es invocada directamente.

B. Cobertura de Ramas (Branches): ~100% Se han diseñado casos de prueba específicos para todos los operadores de decisión lógica condicional (if/else y validaciones de errores):

- **Ramas de validación de argumentos:** Se introducen deliberadamente valores inválidos (negativos o fuera de rangos permitidos para los porcentajes) que activan los bloques throw new Error() de validación.

C. Cobertura de Funciones (Functions): 100% Las 6 funciones exportadas por el archivo (computeIngredientRealCost, computeBruteCostPerPound, computeBruteCostPerKilo, computeRealCostPerPound, computeRealCostPerKilo y computeHydrationAmount) son explícitamente importadas y probadas bajo distintos contextos.

Unit Testing and Code Coverage Report

Module: Backend Service Layer

Testing Technology: Vitest (TypeScript/JavaScript)

Analysis Date: June 2026

1. Function and Business Logic Description

The files under analysis export three service functions responsible for subrecipe versioning, utility tariff management, and sandbox session ingredient modification.

A. `applyVersion(subrecipeId, newVersionId, masterRecipeId)`

Orchestrates the selection of a new version for a subrecipe within a master recipe context.

- **Business rules validated:**
 - If the subrecipe ID does not exist in the registry, the operation returns a `NOT_FOUND` error.
 - If the master recipe ID does not exist, the operation returns a `NOT_FOUND` error.
 - If the requested version ID is not present in the subrecipe's version list, `selectVersion` throws, which is caught and returned as an error.
 - On success, the function computes the cost variation percentage between the previous and new version based on their estimated cost per kilogram.
 - The global `currentVersionId` state is mutated to reflect the newly selected version.
 - Updated total hydration and total cost are recalculated from the scaled ingredient composition.

B. `updateTariffs(updated)`

Validates and persists utility tariff configuration (water, gas, electricity).

- **Business rules validated:**
 - Water price per liter must not be negative.
 - Water additional usage percentage must not be negative.
 - Gas price per hour must not be negative.
 - Electricity price per kWh must not be negative.
 - Electricity oven power must be greater than zero.
 - Electricity fixed surcharge must not be negative.

- If any validation fails, the function returns all accumulated errors without mutating global state.
- On success, global tariff state is updated, and `computeServiceCosts` reflects the new values.

C. `modifyIngredientQuantity(sessionId, ingredientId, newQuantityGrams)`

Modifies an ingredient's quantity within an active sandbox experimentation session.

- **Business rules validated:**
 - If the session ID does not exist, the operation returns a `NOT_FOUND` error.
 - If the session has expired (30 minutes of inactivity), the operation returns an expiration error.
 - If the ingredient ID is not found in the session's modified ingredients list, a `NOT_FOUND` error is returned.
 - On success, the ingredient quantity is updated, the modification is recorded in the history log, and `lastActivityAt` is refreshed.
 - Metrics (`estimatedTotalCost`, `estimatedCostPerUnit`, `totalHydration`) are re-computed and range warnings are generated if hydration falls outside the recommended [40%, 90%] interval.

2. Test Case Matrix

The test suite validates the following scenarios for each service function:

CU_10: `applyVersion` — Subrecipe Version Selection

Test Case	Input Data	Expected Result	Test Type
Subrecipe ID does not exist	<code>("sr_NONEXISTENT", "v1", "fr_001")</code>	<code>{ error: "...no encontrada" }</code>	Error / <code>NOT_FOUND</code>
Master recipe ID does not exist	<code>("sr_001", "v1", "fr_NONEXISTENT")</code>	<code>{ error: "...no encontrada" }</code>	Error / <code>NOT_FOUND</code>
Version ID not found in subrecipe	<code>("sr_001", "v999", "fr_001")</code>	<code>{ error: "Version v999 not found..." }</code>	Error / <code>NOT_FOUND</code>
Happy path — select v2 from baseline v1	<code>("sr_001", "v2", "fr_001")</code>	<code>{ result: { previousVersionId: "v1", costVariationPer</code>	Happy Path

Test Case	Input Data	Expected Result	Test Type
		centage: -13 } }	
Global state mutation in getVersionList	Call getVersionList("sr_001") after selection	currentVersionId === "v2"	State Mutation
Cost variation v2 → v3 (increase)	("sr_001", "v3", "fr_001")	costVariationPercentage === 31	Economic Calculation
Cost variation v3 → v2 (decrease)	("sr_001", "v2", "fr_001")	costVariationPercentage === -24	Economic Calculation
Subrecipe with single version	("sr_002", "v1", "fr_001")	{ result: { newVersionId: "v1" } }	Edge Case

CU_13: updateTariffs — Tariff Validation and Propagation

Test Case	Input Data	Expected Result	Test Type
FA1 — Negative water pricePerLiter	water.pricePerLiter = -1	1 error: field "water.pricePerLiter"	Validation
FA2 — Negative gas pricePerHour	gas.pricePerHour = -500	1 error: field "gas.pricePerHour"	Validation
FA3 — Zero ovenPowerKw	electricity.ovenPowerKw = 0	1 error: field "electricity.ovenPowerKw"	Validation
Negative electricity pricePerKwh	electricity.pricePerKwh = -1	1 error: field "electricity.pricePerKwh"	Validation
Negative fixedSurcharge	electricity.fixedSurcharge = -100	1 error: field "electricity.fixedSurcharge"	Validation
Negative additionalUsagePercentage	water.additionalUsagePercentage = -5	1 error: field "water.additionalUsagePercentage"	Validation
Multiple simultaneous errors	3 invalid fields at once	≥ 3 errors returned	Validation / Aggregation
Valid tariffs update global state	water.pricePerLiter = 2500	tariffs returned, errors undefined	State Mutation
getTariffs() returns new	Update water to 9999	getTariffs().wat	State Persistence

Test Case	Input Data	Expected Result	Test Type
values		<code>er.pricePerLiter === 9999</code>	
Invalid input does NOT mutate state	<code>water.pricePerLiter = -100</code> (invalid)	Previous values preserved	Rollback Safety
Default cost computation (baseline)	Default tariffs + recipe	<code>ingredientsCost: 8635, waterCost: 82800, gasCost: 3570, electricityCost: 2107, subtotal: 88477, totalCost: 97112, costPerUnit: 1619</code>	Happy Path
FA1 propagation — updated water tariff	<code>water.pricePerLiter = 2000</code>	<code>waterCost: 138000, costPerUnit: 2539</code>	Propagation
FA2 propagation — updated gas tariff	<code>gas.pricePerHour = 12000</code>	<code>gasCost: 5040</code>	Propagation
FA3 propagation — updated oven power	<code>electricity.ovenPowerKw = 7</code>	<code>electricityCost: 2749</code>	Propagation
Zero unitsPerBatch edge case	<code>unitsPerBatch = 0</code>	<code>costPerUnit === 0</code>	Edge Case

CU_17: modifyIngredientQuantity — Sandbox Ingredient Modification

Test Case	Input Data	Expected Result	Test Type
Session ID does not exist	<code>("non-existent-session", ...)</code>	<code>{ error: "...no encontrada" }</code>	Error / NOT_FOUND
Session expired (31 min inactivity)	<code>vi.advanceTimersByTime(31 * 60 * 1000)</code>	<code>{ error: "...expirado" }</code>	Error / Expiration
Ingredient ID does not exist	<code>(sessionId, "i_NONEXISTENT", 999)</code>	<code>{ error: "...no encontrado" }</code>	Error / NOT_FOUND
Happy path — modify ingredient	<code>(sessionId, "i_harina", 600)</code>	<code>{ session: { ... } }</code> without error	Happy Path
New quantity value applied	Check after modification	<code>ingredient.quantityGrams === 600</code>	Correctness

Test Case	Input Data	Expected Result	Test Type
Modification recorded in history	Check modifications array	<code>modifications.length === 1</code>	Correctness
Accumulate multiple modifications	3 consecutive modifications	<code>modifications.length ≥ 3</code>	Accumulation
Session stays active after modification	Modify twice consecutively	<code>secondOp.error === undefined</code>	Activity Refresh
Other ingredients preserved	Modify <code>i_harina</code> , check <code>i_sal</code>	<code>i_sal.quantityGrams</code> remains 10	Immutability
Estimated total cost updated	Modify mantequilla 300 → 500	<code>costAfter !== costBefore</code>	Metrics Recalculation
Range warning — hydration below 40%	Reduce <code>i_harina</code> to 20g	<code>rangeWarnings.length ≥ 1</code>	Range Warning
Range warning — hydration above 90%	Increase <code>i_agua</code> to 5000g	<code>rangeWarnings.length ≥ 1</code>	Range Warning
No warnings for normal hydration (58%)	Default recipe (no modifications)	<code>rangeWarnings.length === 0</code>	Edge Case
Estimated cost per unit updated	Modify mantequilla 300 → 500	<code>unitAfter !== unitBefore</code>	Metrics Recalculation

3. Coverage Analysis

Based on the implemented unit test suite, the following technical projection is made regarding the code coverage of the source files associated with the backend services (`subrecipeVersionService.ts`, `utilityCostService.ts`, `sandboxService.ts`).

A. Statement Coverage: ~100%

The tests cover all feasible logical branches and paths of the three service functions. Every variable assignment, return block, and utility function call is directly invoked.

B. Branch Coverage: ~100%

Test cases have been designed for all conditional decision operators:

- **applyVersion:** 3 NOT_FOUND branches (subrecipe, master recipe, version) + success path with economic calculation and state mutation.
- **updateTariffs:** 6 individual validation rules (FA1/FA2/FA3 plus additional rules) + success path + rollback guarantee on validation failure.
- **modifyIngredientQuantity:** Session not found branch, expiration branch, ingredient not found

branch, success path with metrics re-computation and range warning generation (below minimum, above maximum, within normal range).

C. Function Coverage: 100%

All exported functions under test are explicitly imported and invoked:

- `applyVersion` from `services/subrecipeVersionService.ts`
- `updateTariffs`, `getTariffs`, `computeServiceCosts` from `services/utilityCostService.ts`
- `modifyIngredientQuantity`, `openSession`, `getSessionMetrics` from `services/sandboxService.ts`

Report generated on June 2026 — Banneton Project — Backend Service Layer Unit Tests